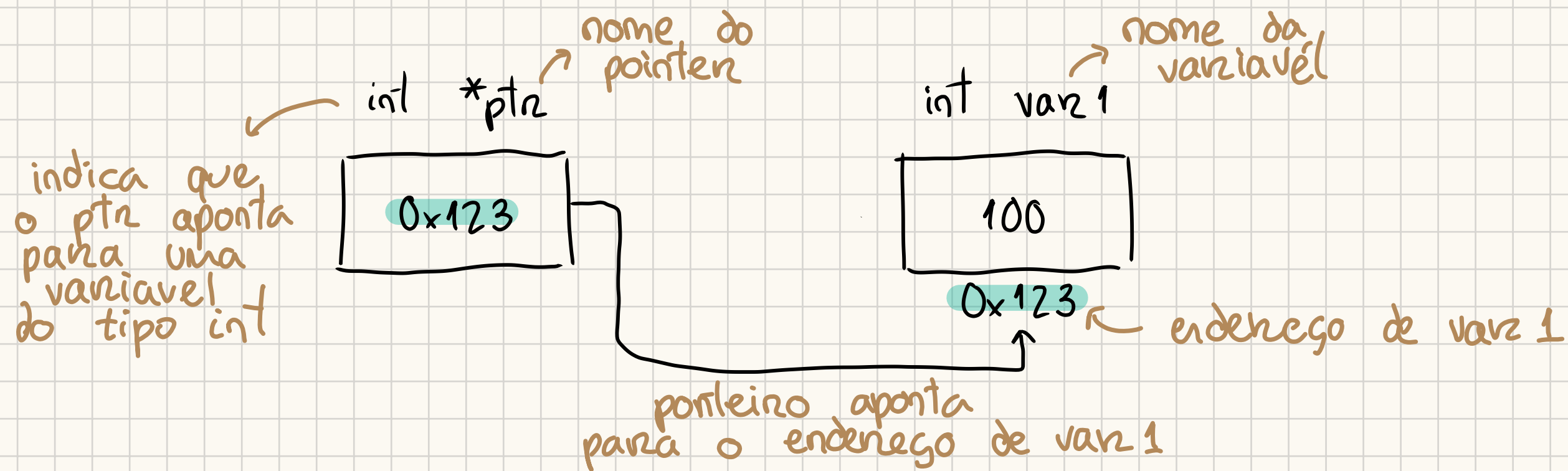


Linguagem C

Ponteiros

Variável que guarda **endereço** de outra variável



`int * ptr = NULL` ← representa um ponteiro não inicializado (permite testar o ponteiro primeiro)

• Operadores

`&` (address operator): devolve o endereço da variável apontada

`*` (dereference operator): acede à variável apontada

• Arrays

```
int my_array[3] = { 1, 2, 3 }
```

`* my_array` devolve "1" → equivalente a `my_array[0]`

Memória

- vezes que se escreve malloc/calloc = vezes que se escreve free

Alocar memória explicitamente

↳ void* malloc (int) → aloca int bytes de memória (sem inicialização)

↳ void* calloc (int, int1) → aloca int blocos de int1 tamanho cada (inicializa cada byte a 0)

Ex: Espaço para um int

```
int* ptr = NULL;
```

```
ptr = (int*) malloc (sizeof(int));
```

```
if (ptr == NULL) { exit(1); }
```

```
...  
free(ptr);
```

→ Alocar memória

→ Caso a alocação falhe

→ Liberar memória

Ex: Espaço para um array

```
int* ptr = NULL;
```

```
ptr = (int*) malloc (3 * sizeof(int));
```

```
if (ptr == NULL) { exit(1); }
```

```
...
```

```
free(ptr);
```

→ Alocar espaço para array de 3 ints

→ Verificar se deu certo

→ Liberar memória alocada

Funções

- **Call-by-value** → Função recebe como argumento uma cópia do valor da variável
→ **Não altera o valor "real"** dessa variável

```
Ex: int sum(int a, int b) {  
    return a + b;  
}
```

```
int m = 10  
int n = 20  
int res = sum(m, n); ← res = 30, m = 10, n = 20
```

- **Call-by-pointer/reference** → Função recebe um ponteiro para a variável
→ **Altera o valor "real"** da variável

```
Ex: void swap(int* p-a, int* p-b) {  
    int temp = *p-a;  
    (*p-a) = *p-b;  
    (*p-b) = *p-a;  
}
```

```
int num1 = 5, num2 = 10;  
int* ptr = &num1;  
swap(ptr, &num2); ← num1 = 10, num2 = 5  
                   ↑  
                 &num1
```

Command-Line arguments

Passar argumentos ao executar um programa (passados para a função main)

↳ **argc**: numero de elementos de argv

↳ **argv**: array de strings(char*) passados para o programa

Ex 1:

"./program.exe" → argc = 1

→ argv[0] = nome do programa (program, neste caso)

Ex 2:

"./program2.exe -windowed -clean" → argc = 3

→ argv[0] = nome do programa (program2)

→ argv[1] = -windowed

→ argv[2] = -clean

Registos (struct)

↳ Agrega campos de vários tipos

Ex:

```
struct student {  
    char* name;  
    int n_mec;  
    float grade;  
};
```

typedef

↳ Define novos tipos (auxiliares)

Ex:

```
typedef struct {  
    struct point c;  
    char* name;  
    ...  
} student;
```

Facilita escrita,
compreensão e
depuração do
código

Inicialização e Acesso aos campos de um registo

```
struct student student_Lina = {"Lina", 123456, 15.0};
```

```
char first_letter = student_Lina.name[0] → acesso aos campos do registo
```

```
student_Lina.grade = 20.0
```

```
struct student* ptr = &student_Lina; → pointer para a instancia student_Lina
```

```
printf("%s\n", ptr->grade); → acesso aos campos do registo com ponteiros
```


Complexidade de Algoritmos

Algoritmos Deterministas



Algoritmos Não-Deterministas

- Se o input não muda o output é sempre o mesmo
- Tipo mais habitual de algoritmo
- Pode devolver valores diferentes para inputs iguais
- Usado para obter soluções aproximadas

Análise da Complexidade

- Tipos de Complexidade:

Tempo de execução / N° de operações executadas: Complexidade Temporal ←

Espaço de memória necessário: Complexidade espacial

Vamos estudar mais

- Tipos de Análise:

Análise experimental - Reconhecendo a:

- 1) Testes Computacionais com diferentes inputs
- 2) Registrar o n° de operações / tempo de execução
- 3) Classificar o algoritmo quanto à análise dos resultados

- Tem em conta que:

O tempo de execução depende de fatores externos (p.ex: hardware)!

Análise formal - "Papel e lápis"

- 1) Escolher uma operação significativa (p.ex.: comparações)
- 2) Obter expressão matemática para o n° de vezes que é efetuada (dependendo do tamanho dos inputs)
- 3) Analisar se a organização dos dados influencia o resultado
- 4) Classificar o algoritmo

Verificar se as conclusões das duas análises são compatíveis

Classes de Complexidade Habituais:

mais rápido $\xrightarrow{\hspace{10em}}$ mais lento

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < 2^n < n!$

constante $\swarrow$ $\searrow$ logaritmico $\nwarrow$ linear $\nearrow$ fatorial

Exemplos:

Ciclos - Quantas iterações?

```
• for (k = 1 ; k < 6 ; k++) {  
    ...  
}
```

$$\rightarrow \sum_{k=1}^5 1 = \underline{\underline{5}}$$

```

• for (i = 0; i < m; i++) {
    for (j = 0; j < n; j++) {
        ...
    }
}

```



$$\sum_{i=0}^{m-1} \left(\sum_{j=0}^{n-1} 1 \right) = \sum_{i=0}^{m-1} n = n \sum_{i=0}^{m-1} 1 = \underline{\underline{m \times n}}$$

Ver Slides 3, página 21

Multiplicação de Matrizes quadradas

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

$n \times n$ $n \times n$ n^2 elementos

3 multiplicações para cada elemento
 9 elementos na matriz resultado
 $3 \times 9 = 27$ multiplicações no total

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        c[i][j] = 0;
        for (int k = 0; k < n; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}

```

$$\sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} \left(\sum_{k=0}^{n-1} 1 \right) \right) = \underline{\underline{n^3}}$$

Algoritmo Cúbico

Generalização

$A(m \times n) \cdot B(n \times p) = C(m \times p) \rightarrow m \times n \times p$ multiplicações efetuadas para chegar à matriz resultante, C
 $O(m \times n \times p)$

Por abuso de linguagem:
Complexidade Cúbica, n^3

Ordens de Complexidade

Ordem	Nome	$T(2N)/T(N)$
1	constante	1
$\log N$	logaritmico	~ 1
N	linear	2
N^2	quadrático	4
N^3	cúbico	8
2^n	exponencial	$T(N)$

Linear Search - Procura Sequencial

Se ordenado

$$B_c(n) = 2$$

$$W_c(n) = (n + 1)$$

$$A_c(n) = \frac{n}{2} + 2$$

Se não ordenado

$$B_c(n) = 1$$

$$W_c(n) = n$$

$$A_c(n) = \frac{n}{2}$$

```
int search(int a[], int size, int x)
{
    for(int i = 0; i < size; i++) {
        if(a[i] == x) {
            return i;
        }
    }
    return -1;
}
```

Binary Search - Decrease and Conquer

$B(n) = 1$ $\longrightarrow$ Quando o primeiro elemento é o valor procurado

$$W(n) = O(\log n) - p = A(n)$$

(número de níveis da árvore
(altura da árvore binária))

```

int binSearch (int a [], int size, int x) {
    int left = 0; int right = n-1
    while (left <= right) {
        int middle = (left + right) / 2;
        if (a[middle] == x) return middle;
        if (a[middle] > x) right = middle - 1;
        else left = middle + 1;
    }
    return -1;
}

```

Selection Sort - Colocar maior em ultimo

Numero de trocas:

- $B(n) = 0$
- $W(n) = n-1$
- $A(n) = n - \ln n$

```

void selectionSort (int a [], int n) {
    for (int k = n-1; k > 0; k--) {
        int indMax = 0;
        for (int i = 1; i <= k; i++) {
            if (a[i] >= a[indMax]) indMax = i

```

Numero de comparações:

- $B(n) = W(n) = \frac{n^2}{2} - \frac{n}{2}$

```

void swap (&a[indMax], &a[k]) {
    int aux = *a[k];
    a[k] = *a[indMax];
    a[indMax] = aux;
}

```

```

    }
    if (indMax != κ) swap(&a[indMax], &a[κ]);
  }
}

```

Bubble Sort - Trocar elementos adjacentes se fora de ordem

Numero de trocas:

- $B(n) = 0$
- $W(n) = \frac{n^2}{2} - \frac{n}{2}$
- $A(n) = \frac{1}{3}n^2 - \frac{1}{6}n$

Numero de comparações:

- $B(n) = n - 1$
- $W(n) = \frac{n^2}{2} - \frac{n}{2}$
- $A(n) = \frac{n^2}{3}$

```

void bubbleSort(int a[], int n) {
  int κ = n; int stop = 0;
  while (stop == 0) {
    stop = 1; κ--;
    for (int i = 0; i < κ; i++) {
      if (a[i] > a[i+1]) {
        swap(&a[i], &a[i+1]);
        stop = 0;
      }
    }
  }
}

```

Insertion Sort - Inserir elementos ordenadamente

Numero de trocas:

- $B(n) = 0$
- $W(n) = \frac{n^2}{2}$
- $A(n) = \frac{n^2}{8}$

Numero de comparações.

- $B(n) = n - 1$
- $W(n) = \frac{n-1}{2} \times (n+2)$
- $A(n) = \frac{n^2}{4} + \frac{7n}{4}$

```
void insertionSort(int a[], int n) {  
    for (int i = 1; i < n; i++) {  
        if (a[i] < a[i-1]) {  
            insertElement(a, i, a[i]);  
        }  
    }  
}
```

```
void insertElem(int sorted[], int n, int elem) {  
    int i;  
    for (i = n-1; i >= 0 && (elem < sorted[i]; i--)  
        sorted[i+1] = sorted[i];  
    sorted[i+1] = elem;  
}
```


Diminuir - para - Reinar

- Resolver 1 subproblema de cada vez
- Lista/Cadeia de chamadas recursivas
- Transformavel em iterativo - usando um ciclo

Dividir - para - Reinar

- Resolver 2 ou mais subproblemas de cada vez
 - Arvore de chamadas recursivas
- Transformavel em iterativo - usando uma stack/pilha

Tornes de Hanoi - $2^n - 1$ movimentos para resolver n discos - $O(2^n)$

MergeSort - Subdividir em 2 metades e organizá-las

$$B_c(n) = W_c(n) = A_c(n) = n \log n$$

TADs

- ↳ ficheiro .h : operações públicas + conteúdo
- ↳ ficheiro .c : implementação + representação interna

Stack / Pilha

- Elementos do mesmo tipo armazenados sequencialmente
- LIFO (Last-In-First-Out)

↳ Consulta e inserção/remoção apenas no topo da pilha

- push() • pop() • peek() • size() • isEmpty() • isFull • init() • destroy • clear

Queue / Fila

- Elementos do mesmo tipo armazenados sequencialmente
- FIFO (First-In-First-Out)

↳ Inserção na cauda e remoção/consulta na frente da fila

- enqueue() • dequeue() • peek() • size() • isEmpty() • isFull() • init() • destroy() • clear()

Deque / Double-Ended Queue

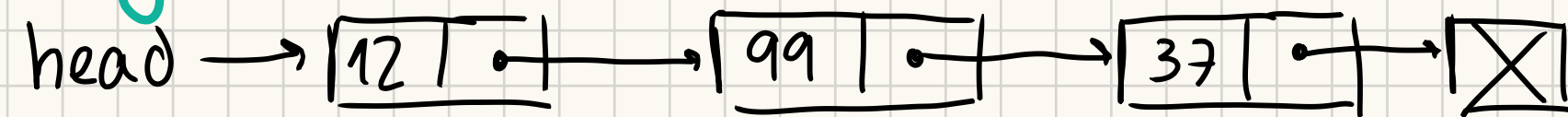
- Elementos do mesmo tipo em ordem sequencial
- Inserção, remoção e consulta nas duas extremidades

Estruturas de Dados Dinâmicas ← Para quando o numero de elementos é variável

Escalabilidade:

- Armazenamento e gestão de coleções de tamanho variável
- Gestão de memória à medida das necessidades

Lista - Ligada



- Cada elemento contém um ponteiro que aponta para o próximo elemento
- Inserção, remoção, substituição e consulta em qualquer posição

Iterar sobre a lista

- Cada nó tem um índice implícito (primeiro nó tem índice 0)
- Associar iterador → ponteiro : current
→ índice : currentPos

Lista Ordenada

Elementos do mesmo tipo

Armazenados em ordem de acord com um critério

↑ Precisa de uma função comparadora

Arvore

- um só caminho entre qualquer par de nós
- arvore com n nós tem $(n-1)$ arcos
- n tem ciclos
- grau da arvore = maior grau de qualquer nó
- nº máximo de nós de nível $i = 2^i$
- nº máximo de nós numa arvore de altura $h = 2^{h+1} - 1$
- nº mínimo de nós numa arvore de altura $h = h + 1$

Arvore Binaria de Procura - ABP/BST

- Para cada nó, os elementos da subarvore esquerda são inferiores e da direita superiores
- Não há elementos repetidos
- Altura depende da ordem de inserção

Complexidade no pior caso:

Procura, Inserção e Remoção - $O(n)$

Arvore Equilibrada

- Altura das subarvores de cada nó difere, no máximo, por 1

Complexidade no Pior caso:

Procura, Inserção e Remoção - $O(\log_2 n)$

Arvore AVL

- Arvore Equilibrada que mantém o equilibrio sempre que se adiciona ou remove nós
- Manter equilibrio com operações de rotação: Simples ou Duplas

Complexidade no Pior caso:

Procura, Inserção e Remoção: $O(\log_2 n)$

Exemplos de código

Ex 1) Contar o nº de nós de uma arvore binária


```

int TreeGetNumberOfNodes(const Tree* root) {
    if (root == NULL) return 0;
    return 1 + TreeGetNumberOfNodes(root → left) + TreeGetNumberOfNodes(root → right);
}

```

2) Destruir uma árvore binária

```

void TreeDestroy(Tree** pRoot) {
    Tree* root = *pRoot
    if (root == NULL) return;
    TreeDestroy(&(root → right));
    TreeDestroy(&(root → left));
    free(root);
    *pRoot = NULL;
}

```

3) Verificar se um item pertence à árvore

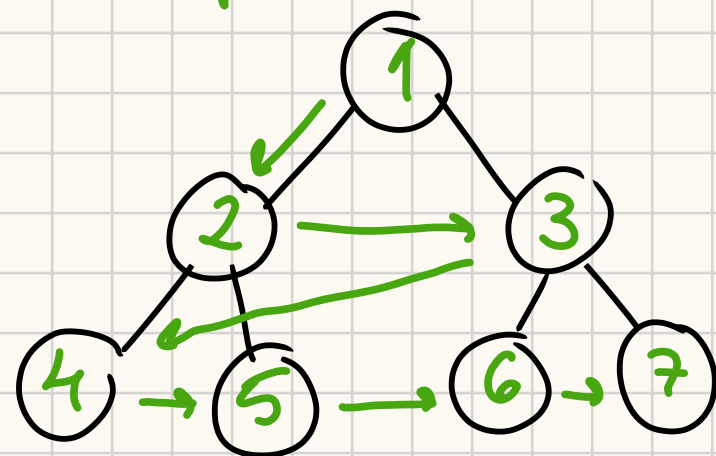
```

int TreeContains(const Tree* root, const ItemType item) {
    if (root == NULL) return 0;
    if (root → item == item) return 1;
    return TreeContains(root → left, item) || TreeContains(root → right, item);
}

```


Travessia

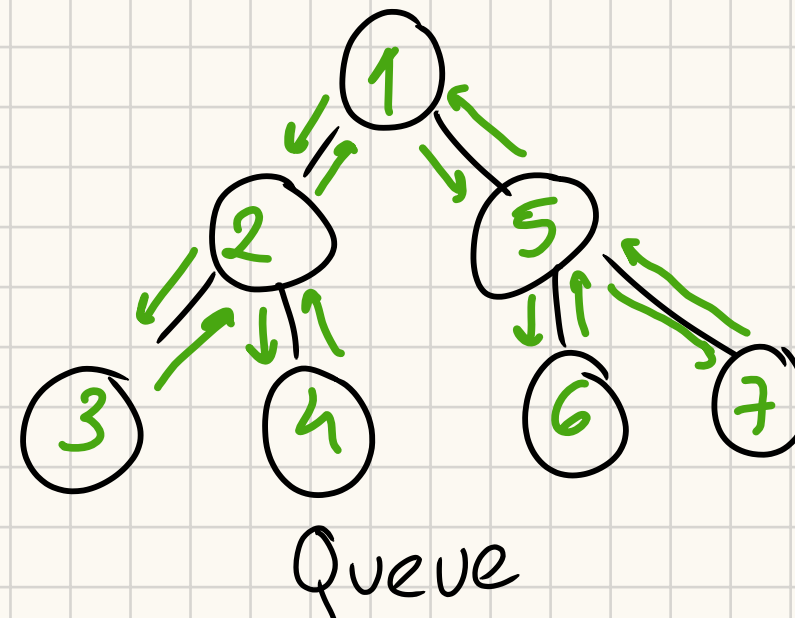
Depth-First



Iterativas:

Stack

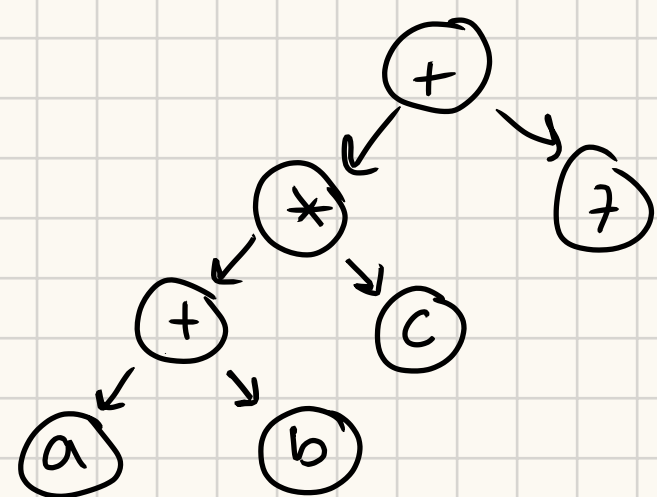
Breadth-First



Queue

Tipos de Travessia

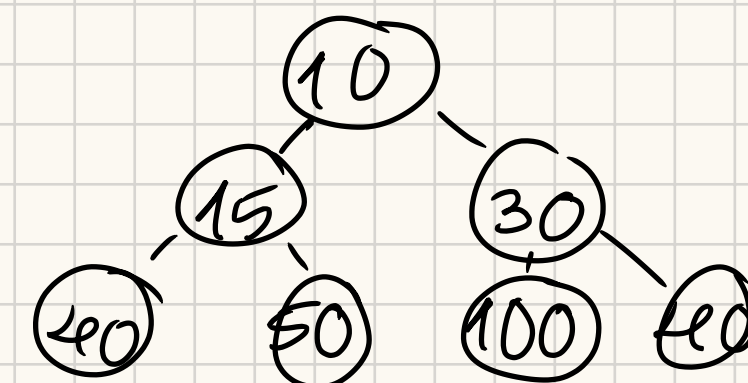
Depth-First



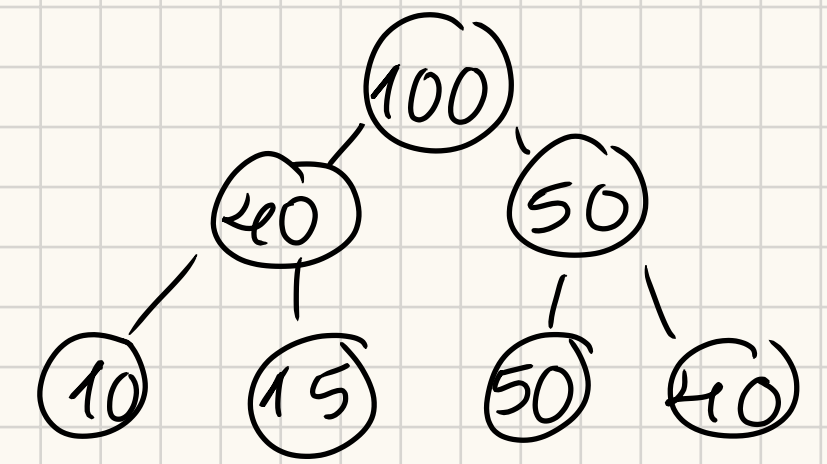
- Pré-ordem (NLR): $+ * + a b c 7$
- Em-ordem (LNR): $a + b * c + 7$
- Pós-ordem (LRN): $a b + c * 7 +$

Binary Heaps → para representar filas com prioridade usando arvores binárias completas

Min-Heap: O valor/chave de um nó não é superior ao dos seus filhos
Sequencia de valores da raiz até uma folha é não-decrescente



Max-Heap: valor/chave de um nó não é inferior ao dos filhos
valores da raiz até uma folha são não-crescentes



Podem existir valores/chaves repetidos!

Eficiência Computacional

Consultar o elemento de maior/menor prioridade - $O(1)$

Adicionar um elemento - $O(\log n)$

Apagar o elemento de maior/menor prioridade - $O(\log n)$

Transform and Conquer - Heap-Sort

Dado um array de n elementos

- 1) Construir um max-heap, que descubra o maior elemento ← Com buildMaxHeap - $O(n)$
- 2) Repetir $(n-1)$ vezes :
 - Levar o maior elemento para a posição final do array ← Com fixHeap - $O(\log n)$
 - remover o maior elemento da árvore e reorganizar como max-heap

Eficiência Computacional - $O(n \log_2 n)$

↳ $n-1$ chamadas a fixHeap

Resumindo algoritmos de procura e ordenação

Quadráticos:

Bubble Sort — swap se fora de ordem

Selection Sort — selecionar o menor elemento e ordenar

Insertion Sort — a [0] é subconjunto. Ir juntando os restantes de forma ordenada

Logarítmicos:

Procura binária — Diminuir para reinar

Merge Sort — Dividir para reinar

Heap Sort — Transformar (em max-heap) para reinar

↑ para ordenar em ordem crescente
(se fosse decrescente seria min-heap)

Dicionários / Tabela de pares (chave, valor)

- ↳ Usar chave para aceder a um item
- ↳ Chaves têm de ser comparáveis
- ↳ Não existem chaves iguais

Operações básicas

- put — registar um par (valor, chave) ou alterar valor
- get — consultar valor associado a uma chave
- contains — verificar se uma chave pertence

Tabelas de Dispensão/Hashing

- Estrutura de dados para armazenar pares (chave, valor)
- Sem chaves duplicadas
- Sem ordem implícita

Ideia Principal

Armazenar um item e, através de uma função de Hashing, calcular o seu índice a partir da chave

Ex:

	Chaves (máx 12)	Código ASCII da primeira letra	$\% 17^*$	Array de 17^* elementos
ordem de inserção	Janeiro	74	6	0
	Fevereiro	70	2	2
	Março	77	9	9
	Abril	65	14	14
	Maio	77	9	10

* número primo maior que o número de chaves (12)

Colisão

Duas chaves diferentes originam o mesmo índice da tabela

Funções de Hashing

Objetivo: Operações quase constantes para qualquer chave

Funções deterministas que transformam a chave - $\text{hash}(n)$

Para uma tabela de tamanho M , com índices de 0 a $M-1$ em que M é um número primo, para saber o índice da tabela onde guardar o valor tem-se:

$$\text{abs}(\text{hash}(n)) \% M$$

Open Addressing - Endereçamento Calculado

↳ Para resolver colisões

Quando há uma colisão armazenar o item no próximo espaço vazio

O tamanho da tabela (M) tem de ser maior (pelo menos o dobro) do que o número de itens

• **Linear Probing** - Tentar em cada $i+1$ - Para Inserir e Procurar na tabela

1) Aceder ao elemento de índice i

2) Se necessário, tentar $(i+1) \% M, (i+2) \% M, \dots$

Problema - Clustering

A forma como chaves são indexadas cria grandes "clusters" ← pouco eficaz

• **Quadratic Probing** - tentar a cada $(i+1)^2$

1) Aceder ao elemento de índice i

2) Se necessário, tentar $(i+1) \% M, (i+4) \% M, (i+9) \% M, \dots$

Redução do efeito de clustering!

Problema

Com Linear Probing tem-se um fator de carga habitual de 50%.

M muito grande → demasiados espaços vazios

M muito pequeno → tempo de procura muito elevado

Solução - Resizing + Reshaping

Quando fator de carga $\geq \frac{1}{2}$ → Duplicar o tamanho do array

Quando fator de carga $\leq \frac{1}{8}$ → Reduzir o tamanho do array para metade

Como?

Criando uma nova tabela e adicionar todos os itens, um a um

Problema

Normalmente quando se apaga um par (chave, valor) de uma hash table prejudicamos buscas futuras, que param ao chegar a um espaço vazio

Ex: Procurar "H" (id = 4)

Tabela M

0	1	2	3	4	5	6	7	8	9	10
P	M			A	C	S	H			E

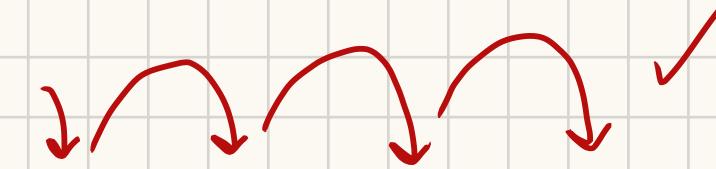


Tabela M depois de apagar "S"

0	1	2	3	4	5	6	7	8	9	10
P	M			A	C		H			E



como o índice 6 está vazio a procura para e falha

Solução - Lazy Deletion

- Marcar todos os elementos como "livres"
- Ao inserir um item - esse elemento fica "ocupado"
- Elementos apagados são, em vez disso, marcados como tal

Uma procura termina ao encontrar a chave procurada ou um elemento marcado como "livre"

Hash Table - Eficiência

Procura, Inserção e Eliminação - $A(n) \in O(1)$, $W(n) \in O(n)$

Separate Chaining - Alternativa ao Open Addressing

- Cada chave inserida, caso já exista na tabela, é inserida na lista ligada correspondente
- Cada posição corresponde a uma lista ligada

Eficiência - $O(1)$

Resizing + Reshaping

Quando $N/M \geq 8 \rightarrow$ Duplicar

Quando $N/M \leq 2 \rightarrow$ Reduzir para metade

Separate Chaining vs Linear Probing

😊 Desempenho não se degrada

😞 Pouco sensível a funções de hashing piores

😊 Menos desperdício de espaço de memória

Gráficos

Passeio - sequência de vértices adjacentes $\rightarrow$ comprimento = $n-1$ arestas

Trajeto - Passeio com arestas distintas

- **Circuito** quando começa e acaba no mesmo vertice

Caminho - Passeio com vertices e arestas distintas

- **Ciclo** quando começa e acaba no mesmo vertice

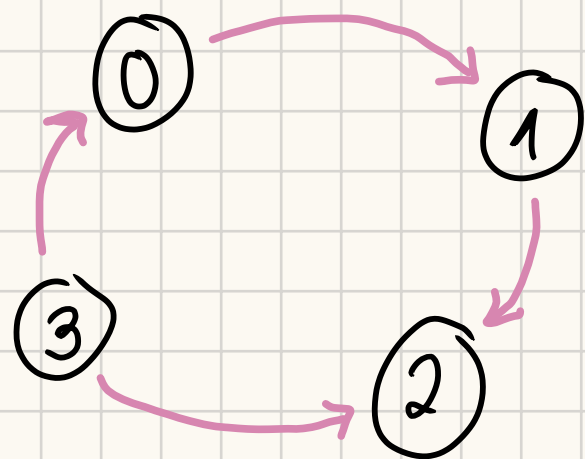
Grafos Orientados

- Cada vertice tem in-Degree e out-Degree

- $N = \text{máximo de arestas} = V \times (V - 1)$ - **Grafo Orientado completo**
 - Quando um vertice não tem in-degree é um vertice fonte/source
 - Quando um vertice não tem out-degree é um vertice sumidouro/sink

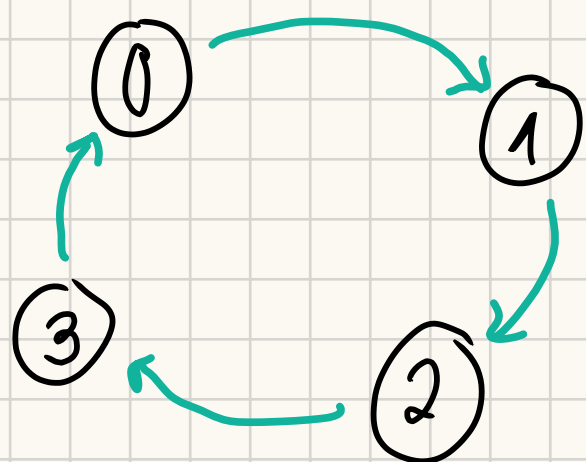
Conectividade

Grafo Orientado fracamente conexo



- Ao substituir as arestas orientadas por arestas não orientadas o grafo resultante é conexo

Grafo Orientado fortemente conexo



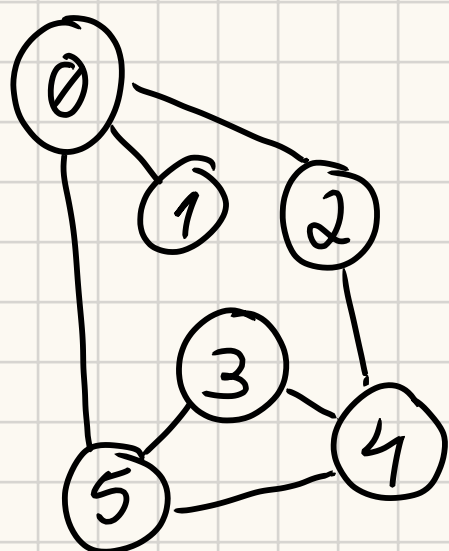
- Existe um caminho entre cada par de vertices
- Atinge-se mais nós ao partir de qualquer nó

Rede - Grafo, orientado ou não, com pesos associados às arestas

TAD Grafo

- vértices representados por um valor (de 0 a $V-1$)

Representação de:



Lista de arestas

0 - 2
0 - 1
0 - 5
1 - 0
2 - 0
2 - 4
...
5 - 3

e

Lista ordenada de arestas

0 - 1
0 - 2
0 - 5
3 - 4
3 - 5
4 - 5

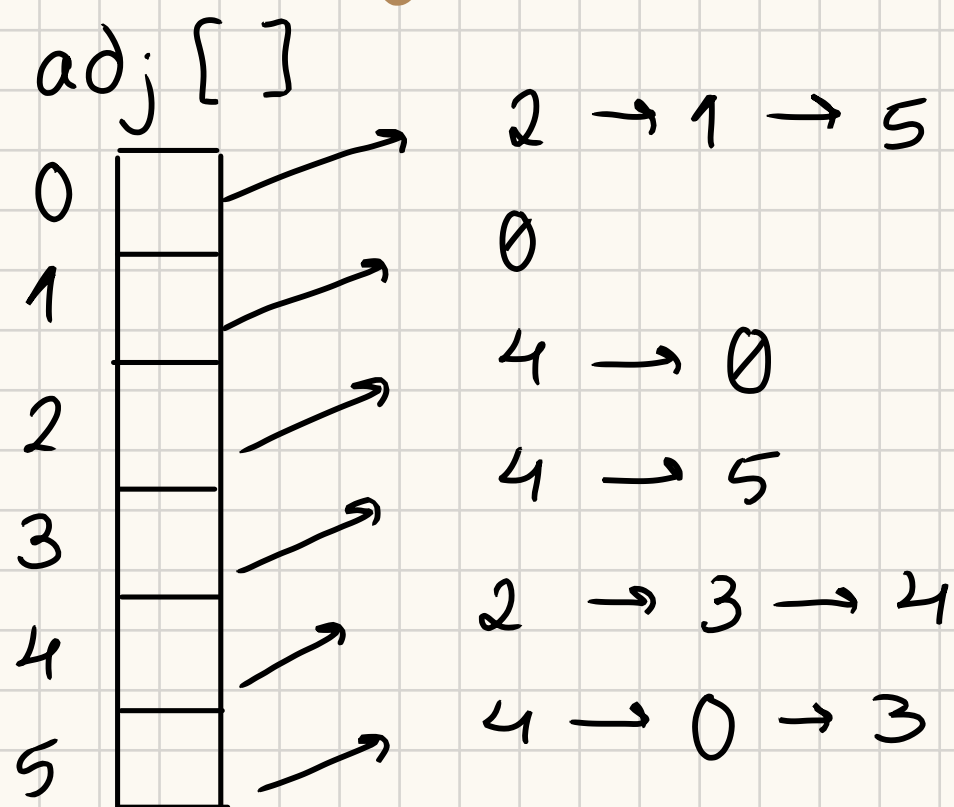
Usando uma lista ligada de arestas

Matriz de adjacências

	0	1	2	3	4	5
0	0	1	1	0	0	1
1	1	0	0	0	0	0
2	1	0	0	0	1	0
3	0	0	0	0	1	1
4	0	0	1	1	0	1
5	1	0	0	1	1	0

representa a aresta entre os vértices 5 e 0

Listas de adjacências



↑ Cada item da lista aponta para uma lista dos seus vizinhos adjacentes

Eficiência das Representações - Grafo (orientado) com V vértices e E arestas

	Espaço	Adicionar aresta	Encontrar aresta entre v e w	Iterar sobre vértices adjacentes a v
• Lista de arestas	E	1	E	E
• Matriz de adjacências	V^2	1	1	V
• <u>Listas de adjacências</u> preferível na prática	$E + V$	1	(out) degree (v)	(out) degree (v)

Travessia

Depth-First Traversal

↳ Recursivo ou Iterativo com **stack**

Util para : encontrar caminho entre 2 vértices / identificar vértices alcançáveis

Idêntico à travessia em profundidade de uma árvore binária mas:

↳ Há um vértice inicial - start vertex

↳ Para cada vertice, o número de vertices adjacentes é variável

↳ Para não entrar em ciclo marcar vertices já visitados (com array de bools por exemplo)

Pseudocódigo - Recursivo :

Travessia em Prof (vertice v)

marcar v como visitado

Para cada vertice w adjacente a v

Se w não está marcado

Então $\text{travessia em Prof}(w)$

Pseudocódigo - Iterativo :

Travessia em prof (vertice v)

criar um stack vazio

$\text{push}(\text{stack}, v)$

marcar v como visitado

enquanto não vazio (stack) fazer

para cada vertice w adjacente a v

se w não está marcado como visitado

então $\text{push}(\text{stack}, w)$

marcar w como visitado

Breadth-First Traversal

↳ Iterativo com queue

Útil para : Determinar caminhos mais curtos

Idêntico à travessia por nível de uma árvore binária

Igual à travessia em profundidade de um grafo mas com **queue**

Pseudocódigo - Iterativo:

Travessia por níveis (vértice v)

criar fila vazia

enqueue (fila, v)

marcar v como visitado

enquanto não vazio (fila) fazer

$v = \text{dequeue}(\text{fila})$

para cada vértice w adjacente a v

se w não está marcado como visitado

então enqueue (fila, w)

marcar w como visitado

Ordenação Topológica

Objetivo: Representar um grafo orientado com orientação com ordenação das ordens de precedência

- útil para verificar se o grafo é acíclico

1) Procurar vértice sem arestas incidentes e adicionar à ordem topológica

2) Apagar esse vértice do grafo original

3) 1)

- Podem existir várias ordenações válidas para o mesmo grafo

Algoritmos

1) Cópia de G

Criar G'

enquanto possível:

selecionar um vértice sem arestas incidentes

imprimir o seu ID

apagar esse vértice de G' e as arestas que dele emergem

Prós:

- Usar in-degree

Contras:

- Gasto de tempo e memória

- Ineficaz

↳ criar cópia

↳ procura sucessivas

2) Array auxiliar

Registrar num array `numEdgesPerVertex` o InDegree de cada vértice

enquanto possível:

selecionar vértice v com `numEdgesPerVertex[v] == 0` e não marcado

imprimir o seu ID

marcá-lo como pertencendo à ordenação

para cada vértice w adjacente a v :

`numEdgesPerVertex[w] --`

Ineficiência: procura sucessivas através do conjunto de vértices

3) Manter candidatos e usar fila

Registrar num array `numEdgesPerVertex` o InDegree de cada vértice

criar fila vazia e inserir os vértices v com `numEdgesPerVertex[v] == 0`

enquanto a fila não estiver vazia:

v = retirar próximo vértice da fila

Imprimir o seu ID

para cada vértice w adjacente a v :

`numEdgesPerVertex[w] --`

se `numEdgesPerVertex[w] == 0` então inserir w na fila

Problemas NP

- Problemas de resposta sim ou não
- Problemas **resolvíveis** em tempo polinomial por uma máquina **Turing não-determinista**
- Problemas **verificáveis** em tempo polinomial ^{ou} por uma máquina **Turing determinista**

NP-Complete : Grupo de problemas NP que, ao descobrir como resolver* um deles, descobre-se como resolver os restantes problemas

NP-Hard : Grupo de problemas que, ao descobrir como resolver* um deles, descobre-se como resolver qualquer problema NP

* em tempo polinomial

Problema do Caixeiro Viajante - NP Difícil

Determinar o caminho mais curto que atravessa n cidades, visitando cada apenas uma vez e acabando na inicial

Solução

- Representar através de um grafo, com cidades associadas aos vértices e distâncias às arestas
- Determinar o ciclo Hamiltoniano mais curto
 - ↑ ciclo que passa por todos os vértices, sem repetições e que acaba no vértice inicial

Como fazer

- 1) Escolher um vértice como inicial
- 2) Gerar $(n-1)!$ permutações dos vértices intermédios
- 3) Para cada ciclo (possível solução) calcular o seu custo total
- 4) Guardar o ciclo mais curto

Desempenho - $O(n!)$

Soma dos Subconjuntos - Subset Sum - NP-Completo

Dado um conjunto de n inteiros, existe um subconjunto cuja soma seja n ?

Solução:

- 1) Gerar todos os subconjuntos
- 2) Verificar a soma dos elementos de cada subconjunto e ver se corresponde a n

O Problema da Mochila - 0-1 Knapsack Problem - NP-Difícil

Determinar o subconjunto de itens mais valioso que cabe numa mochila de capacidade w

Solução:

- Gerar 2^n subconjuntos de n itens
- Para cada calcular o seu peso
- Guardar o mais valioso que cabe na mochila

Desempenho - $O(2^n)$

Quadrado Mágico

A soma dos elementos de cada linha = soma de cada coluna = soma das duas diagonais principais
Quantos quadrados existem para um dado n ?

Solução:

- Gerar cada uma das permutações do conjunto de n^2 itens
- Verificar se cada permutação satisfaz as restrições
- Imprimir cada permutação admissível

Desempenho - $O(2^n)$

Diferenças de C++

- tipo bool
- if, switch
- call by reference
- linguagem OO (classes, herança, polimorfismo)
- programação genérica
- try-throw-catch

Output/Input

```
#include <iostream>
```

```
#include <iomanip>
```

```
#include <math.h>
```

```
std::cout << "enter int? " << endl;
```

```
std::cin >> n;
```

```
std::cout << setw(2) << n
```

```
    << setprecision(15) << sqrt((double)n)
```

```
    << endl;
```

Name Spaces

Para "guardar" variáveis e funções quando se utiliza o mesmo nome em diferentes contextos

```
namespace New {  
    int n = 10; }
```

```
cout << New::n; ← 10
```

```
namespace Newer {  
    int n = 42; }
```

```
cout << Newer::n; ← 42
```

Funções

- Argumentos podem ser passados por valor, por ponteiro e **por referência**
 - ↑ altera valor apenas dentro da func
 - ↑ altera valor fora da func

```
void swap(int &x, int &y) {  
    int tmp = x;  
    x = y;  
    y = tmp;  
}
```

```
result = swap(x, y);
```

Argumentos por omissão (Apenas ultimos!)

Inicialização habitualmente no protótipo da função

prog.h:

```
int f(int x, int y = 2, int z = 3)
```

prog.c:

```
int f(int x, int y, int z) {  
    ...  
}
```

Overloading

- Funções com mesmo nome mas **argumentos diferentes**
- Não se podem distinguir apenas pelo resultado

~ obrigatório

Ex:

```
int square (int n) { return n * n; }  
double square (double n) { return n * n; }
```

• Funções genéricas - Template Functions

- Funções definidas sem especificar os tipos de todos os argumentos ou resultado
- Definindo uma família de funções

```
template < typename T > T f (T n) {  
    return T(7) * n;  
}
```

• auto

tipo da variável é automática/ deduzido a partir do seu valor inicial

Gestão de memória

- new em vez de malloc / calloc
- delete / delete [] em vez de free

Containers

Array:

- tamanho fixo : .size()
- aceder a uma dada posição : .at()
- aceder ao primeiro ou ultimo elem : .front() , .back()

Vector:

- Generalização do array
- tamanho variável
- métodos eficientes para operações na cauda: `v.push-back(3)`, `v.pop-back()`
- `v.insert(v.begin(), 6)` e `v.erase(v.begin())`
 - ↑ insere elemento antes do primeiro
 - iterador

Deque

- tamanho variável
- métodos eficientes para operações na frente e na cauda: `d.push-front(99)`, `d.push-back(6)`
- `d.empty()`

Container Adaptor

Stack

- LIFO
- tamanho variável
- acesso só ao último inserido

Queue

- FIFO
- tamanho variável
- adicionar na cauda e retirar da frente

Associative Containers

- ↳ Procura eficiente - $O(n \log n)$
- ↳ Estruturas de dados ordenadas de tamanho variável

Map

- sem chaves repetidas
- valores de itens podem ser alterados
- valores das chaves são constantes
- ordenado de acordo com as chaves

Set

- sem repetições de elementos
- valores não podem ser alterados
- Não insere o elemento se este já existir
- ordenado de acordo com o valor dos elementos

Resumindo Containers

Array: Tamanho fixo

Vector: Array mas com tamanho variável (automaticamente)

Deque: Vector mas com operações eficazes nas extremidades

} Sequence containers

Stack: LIFO com tamanho variável

Queue: FIFO com tamanho variável

} Container Adaptors

Map: Coleção de pares (chave - valor) ordenados pela chave e sem chaves repetidas

Set: Coleção de elementos ordenados pelo seu valor e sem repetições

} Associative Containers

Iteradores

- Permitem acessar a elementos de containers
- Forward, Bidirectional e Random access iterators

Lambda Expressions - Funções Anônimas

↳ Funções pequenas escritas diretamente no código ou como argumento de outra função

Sintaxe: `[capture] (parameters) { code }`

- ↳ by value ou by reference
- ↳ Acesso a variáveis externas

Standard Algorithms Library

- Disponibiliza funções que operam sobre sequências de elementos, usando iteradores

Exemplos

- **count**: `int n = count(arr.begin(), arr.end(), 3);` ← Conta quantos elementos iguais a 3
- **count_if**: `int n = count_if(arr.begin(), arr.end(), isEven);` ← Conta elementos pares (usando uma função auxiliar, `isEven`)
- **all_of**: `if (all_of(arr.begin(), arr.end(), [](int i) { return i % 2 == 0; })) { ... ; }`
↳ função lambda
- **equal**: `bool b = equal(arr1.begin(), arr1.end(), arr2.begin());` ← Compara arrays, seguindo o range do `arr1`!
- **max_element**: `auto e = max_element(begin(elems), end(elems));`
- **find**: `auto it = find(v.begin(), v.end(), 6)` ← "it" é um pointer para o elemento 6, se não existir aponta para `v.end()`!

- **binary_search**: `if (binary_search(v.begin(), v.end(), 3)) cout << "found it";` ← "v" tem de estar ordenado
- **copy**: `std::copy(vect_orig.begin(), vect_orig.end(), vect_dest.begin());` ← vect_dest tem de ter espaço alocado suficiente
- **sort**: `sort(v.begin(), v.end());` ← Por omissão ordena usando "<", em ordem crescente
- **transform**: `transform(str.begin(), str.end(), str.begin, [](char c) {return toupper(c);})`
(função lambda)